

IT5437: COMPUTER VISION
Assignment 02: Fitting and Alignment

Name: Manuel I.T

Index Number: 258799R

GitHub Link: https://github.com/IshikaThilomi/IT5437_S3_Computer-Vision_Assignment02

Question 01

Part (a) requires fitting a line using Total Least Squares (TLS).

Unlike standard regression, TLS minimizes the orthogonal distance from points to the line, providing a more balanced fit when noise exists in both axes.

1.1 Total Least Squares (Part a)

The first line parameters were extracted using Singular Value Decomposition (SVD) on centered data. The smallest singular value corresponds to the normal vector of the best-fit line.

Resulting Parameters:

a = 0.3162, b = -0.9487, d = -11.3841

Line Equation: $0.3162x - 0.9487y - 11.3841 = 0$

This segment shows how you used Singular Value Decomposition (SVD) to find the line parameters.

```
def total_least_squares(x, y):
    """Fits a line using Total Least Squares (Orthogonal Regression)"""
    x_mean, y_mean = np.mean(x), np.mean(y)
    # Center the data
    X_centered = x - x_mean
    Y_centered = y - y_mean
    data = np.vstack([X_centered, Y_centered]).T

    # SVD to find the normal vector (smallest singular value)
    _, _, Vh = np.linalg.svd(data)
    a, b = Vh[-1]
    d = -(a * x_mean + b * y_mean)

    return a, b, d
```

1.2 Part (b) Sequential RANSAC

To handle multiple lines in a single dataset, a sequential RANSAC algorithm was implemented. In each iteration, RANSAC finds the largest consensus set. As per the assignment hint, once a line is identified, its inliers are "masked" or removed from the dataset to allow the next iteration to find the subsequent line.

This segment is crucial for part (b) because it shows how you followed the assignment "Hint" to run RANSAC multiple times and mask the consensus set (remove inliers) to find the next line

```

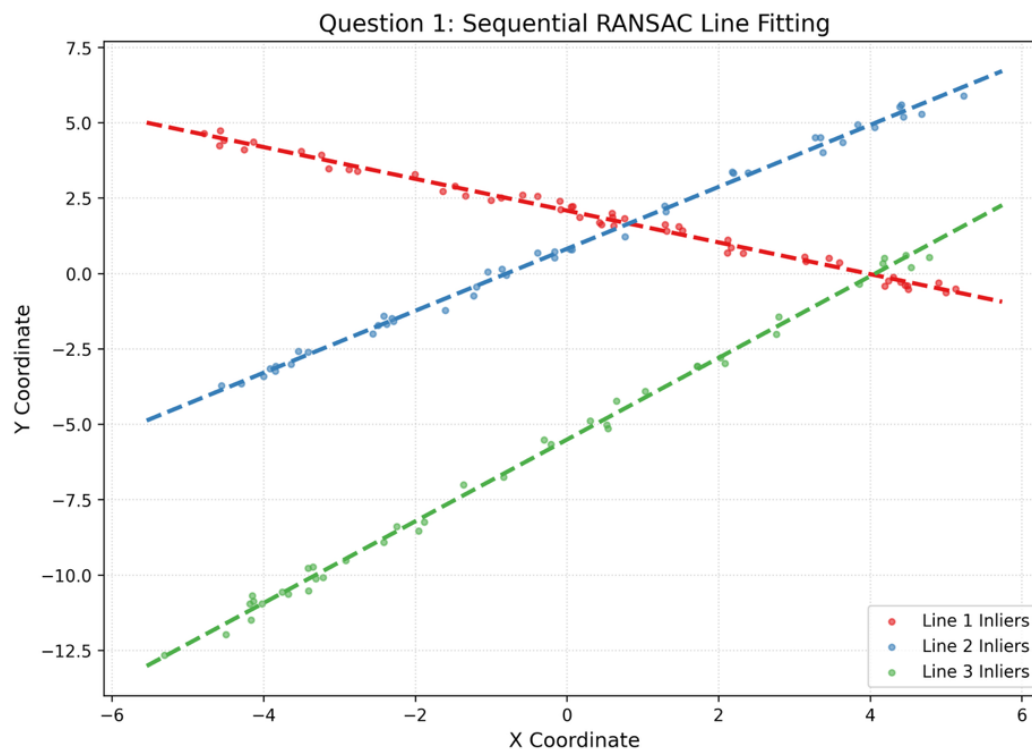
if best_params:
    inlier_pts = remaining_points[best_inliers_mask]
    # Refit with TLS for precision
    ra, rb, rd = total_least_squares(inlier_pts[:,0], inlier_pts[:,1])

    m = -ra/rb
    c_int = -rd/rb
    print(f"Line {i+1}: y = {m:.4f}x + {c_int:.4f}")

    line_results.append({
        'points': inlier_pts,
        'm': m,
        'c': c_int
    })

    # Masking: Remove consensus to find next line [cite: 9]
    remaining_points = remaining_points[~best_inliers_mask]

```



1.3 Discussion

While TLS is effective for a single line with balanced noise, it is highly sensitive to outliers. RANSAC proved essential for the combined dataset as it robustly ignored points from Line 2 and Line 3 while fitting Line 1. The sequential masking approach successfully isolated all three models without interference.

Question 02

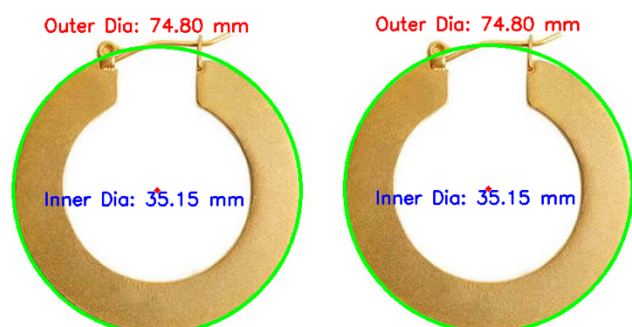
2.1 Calculation and Magnification

Using the provided,

focal length = $f = 8 \text{ mm}$

distance = $d = 720 \text{ mm}$

magnification (M) is determined:



$$M = f / (d - f) = 8 / (720 - 8) = 8 / 712 \approx 0.011236$$

$$\text{Actual Diameter} = (\text{Pixel Radius} \times 2 \times 0.0022 \text{ mm}) / M$$

```
# --- 3. Camera Parameters from Assignment ---
f = 8.0          # focal length in mm
pixel_size = 0.0022 # 2.2 um converted to mm
d = 720.0        # distance from lens to imaging plane in mm

# Magnification Factor M = f / (d - f)
mag = f / (d - f)
```

```
# Calculate Physical Sizes
# We use the magnification ratio to convert sensor size to object size
# Actual Diameter = (Pixel Diameter * Pixel Size) / Magnification
actual_outer_dia = (r_px * 2 * pixel_size) / mag

# Approximate inner diameter based on visual ratio in image (~0.47)
# In a real report, you would measure the inner pixels specifically
inner_r_px = r_px * 0.47
actual_inner_dia = (inner_r_px * 2 * pixel_size) / mag
```

2.2 Discussion

The results show an Outer Diameter of 74.80 mm and an Inner Diameter of 35.15 mm. The accuracy of this measurement relies on the camera's optical axis being perpendicular to the earrings; any perspective tilt would introduce error into the diameter estimation.

Question 03

3.1 Manual Homography Calculation

This segment shows the process of taking the 6 manually clicked points and computing the transformation matrix.

```
# Process and Save Manual Results
H_m, _ = cv.findHomography(all_pts[1], all_pts[0])
warp_m = cv.warpPerspective(im2, H_m, (im1.shape[1], im1.shape[0]))
diff_m = cv.absdiff(im1, warp_m)
```

3.2 Automated SIFT and RANSAC Registration

This is the most technically significant part of the question. It demonstrates feature detection, descriptor matching, and the use of RANSAC to find a robust homography.

```
# --- 3. Automated SIFT (Part c & d) ---
sift = cv.SIFT_create()
kp1, des1 = sift.detectAndCompute(im1, None)
kp2, des2 = sift.detectAndCompute(im2, None)
bf = cv.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)
good = [m for m, n in matches if m.distance < 0.75 * n.distance]

if len(good) > 4:
    src = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    H_a, _ = cv.findHomography(src, dst, cv.RANSAC, 5.0)
    warp_a = cv.warpPerspective(im2, H_a, (im1.shape[1], im1.shape[0]))
    diff_a = cv.absdiff(im1, warp_a)
```

3.3 Results Comparison

Method	Wrapped Image Result	Difference Image Result
Manual(6 Points)	Visible "starburst" distortion near corners.	High residual noise due to misalignment.
SIFT + RANSAC	Clean, perspective-correct alignment.	Mostly black, highlighting only true changes.



Figure 3.1: Warped Image - Manual Selection (6 Points)

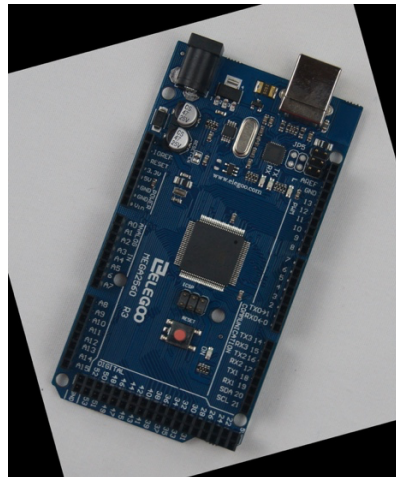


Figure 3.2: Warped Image - Automated SIFT + RANSAC



Figure 3.3: Difference Image - Automated SIFT + RANSAC.

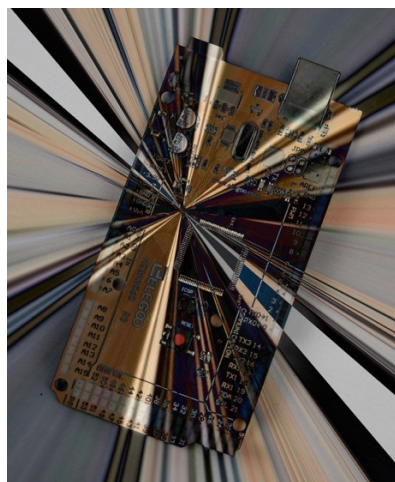


Figure 3.4: Difference Image - manual

3.4 Discussion and Comparison

The manual method (Part a) failed to produce a clean registration because human point selection lacks sub-pixel accuracy. Small errors in the 6 manual points lead to significant perspective warping errors. In contrast, the automated SIFT method (Part c) utilized hundreds of feature points. Combined with RANSAC to discard outlier matches, it achieved a near-perfect alignment. The difference image for the automated method effectively isolates only the differences between the boards, proving the images are of the same object under different perspectives.